

Programming Languages Final Üşengeçlere Özel Çalışma Notu ve Soru Bohçası

Prepared by H.M.V, O.K.U

bu notlarla calisip finale girdim. hoca 25 puanlik lisp kismini bizim sene farkli sormustu(22-23).kalan 75 puanlik kisimdan 46 aldım..

PL Old Final Questions

21-22 Final

1. What are the language evaluation criteria?

Readability: Most important

- Overall simplicity
 - Too many features is bad
 - Multiplicity of features is bad
- Orthogonality
 - Makes the language easy to learn and read
 - Meaning is context independent
- Control statements
- Data type and structures – Syntax considerations

Writability:

- Simplicity and orthogonality
- Support for abstraction
- Expressivity

2. What is the relationship between inheritance and sub-typing?

In the object-oriented framework, inheritance is usually presented as a feature that goes hand in hand with subtyping when one organizes abstract datatypes in a hierarchy of classes. However, the two are orthogonal ideas.

- Subtyping refers to compatibility of interfaces. A type B is a subtype of A if every function that can be invoked on an object of type A can also be invoked on an object of type B.
- Inheritance refers to reuse of implementations. A type B inherits from another type A if some functions for B are written in terms of functions of A.

3. What are the advantages and disadvantages of functional programming?

Advantages:

Simple semantics, concise, flexible. “No” side effect. Less bugs.

Disadvantages:

Execution efficiency, more abstract and mathematical, thus more difficult to learn and use

5. What is the difference between abstract classes vs interfaces?

- Abstract classes allow you to provide some implementation, while interfaces only provide the signature.
- Abstract classes can have fields, while interfaces cannot have fields.
- A class can extend only one abstract class, but can implement multiple interfaces.

6. What is the difference between an abstract class with pure virtual function in C++ vs interfaces in Java?

Interfaces in Java are similar in that they cannot be instantiated, but they can only have abstract methods (i.e. methods without a default implementation) and no member variables. In C++, an abstract class with pure virtual functions can be thought of as a sort of "interface" that also allows for some implementation, whereas interfaces in Java are purely abstract.

7. What is the difference between procedures and functions? Write 3 examples.

Function: no side effect, return a value, done with expression

Procedure: side effect, executed for it. No return value. Done with statement

C: printf is procedure.

C++: void type is procedure

C++: String.length() is function.

8. What is a module and how is it useful for ADTs?

A program unit with a public interface and a private implementation; all services that are available from a module are described in its public interface and are exported to other modules, and all services that are needed by a module must be imported from other modules.

A module can be reused in any way that its public interface allows.

A module implementation can change without affecting the behavior of other modules.

A module offers general services, which may include types and operations on those types, but are not restricted to these. In addition to ADT, module supports structuring of large programs: Separate compilation and name control.

9. What does Java use instead of modules, and how is it different from modules? (Answer: Java uses packages)

Java uses packages. Java packages don't have a clear boundary for visibility and encapsulation, unlike modules.

10. What is the advantages of dynamic scoping over static scoping?

-convenience(no need to pass variables)

-flexibility since the scope of a variable can change at runtime.

11. Java does not allow static binding, compared to C++. Advantages and disadvantages?

Dynamic Advantages: flexibility (generic program units)

Dynamic Disadvantages: Speed penalty for type checking and interpretation .Type error

detection by the compiler is more difficult.

12. How smalltalk handles multiple inheritance?

Smalltalk handles multiple inheritance by allowing a class to compose multiple other classes and inherit behavior from them, through a mechanism called composition-based inheritance, which is more flexible and less prone to ambiguity than traditional multiple inheritance.

13.1. Bir dilin kuralları verilmiş. Variable'lar yazılmış, function'lar yazılmış. Küçük bir program verilmiş. Output ne verir?

13.2. Write a CFG for how the language handles function. Write the parsing tree for the function.

14. Link listed verilmiş. Write an algebraic specification for the following ADT. Using the following operations: is_element_of(x, L); is_empty(L); create(L); insert(x, L); remove(x,L).

Operations:

Create : L

Insert: L,x -> L

Remove: L,x -> L

is_element_of: L,x -> boolean

is_empty: L -> boolean

15. 3 şıklı Lisp sorusu. Binary Search Tree verilmiş; List içerisinde node var (bunun içerisinde de sayılar). Left tree list olarak, right tree list olarak. Search fonksiyonu, insert fonksiyonu ve remove fonksiyonu yazın. Loop kullanmak, mapping kullanmak yasak. Basic operationlar kullanılabilir.

```
(defun search (x tree)
  (if (null tree)
      nil
      (let ((value (second tree))
            (left-subtree (first tree))
            (right-subtree (third tree)))
        (if (= x value)
            tree
            (if (< x value)
                (search x left-subtree)
                (search x right-subtree))))))

(defun insert (x tree)
  (if (null tree)
      (list nil x nil)
      (let ((value (second tree))
            (left-subtree (first tree))
            (right-subtree (third tree)))
        (if (= x value)
            tree
            (if (< x value)
                (list (insert x left-subtree) value right-subtree)
                (list left-subtree value (insert x right-subtree))))))
```

16.1. Prolog sorusu. n^{th} Fibonacci sayısı çıkaran bir program istenmiş. 3'üncü Fibonacci sayısı 2 döndürmesi lazım (1 1 2). 7'nci sayı 13 döndürmeli (1 1 2 3 5 8 13).

```
fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(N, Result) :-
  N > 1,
  N1 is N-1, fibonacci(N1, R1),
  N2 is N-2, fibonacci(N2, R2),
  Result is R1 + R2.
```

16.2. Fibonacci için bir tane query verilmiş. Senin yazdığın programa göre o query nasıl parse'lanır?

17. What is a Predicate, function, Horn clause? Explain.

Horn Clause: a fragment of first-order logic

Boolean functions (true/false) . Can have arguments. (e.g. `parent(X,Y)`).

Functions: non-Boolean functions (`successor(X)`).

2017 Final

1. Yeni bir dilin özelliklerini tanımlamıştı. Variable, number, expression, function nasıl tanımlanır tanımlamadaki syntax kuralları nedir vermişti. Prefix notation olan bir dildi.

Bu dilde number oluşturmayı gösteren dfa'yı çizmemizi istemişti. Sayılar integer // integer şeklinde oluyordu, leading bir 0 olamıyordu, "//" ifadesinden sonra whitespace gelemiyordu. (1//3 , 11//51 gibi)

Bu dil için BNF tanımlamamızı istiyordu. Terminal semboller <expr_func> şeklinde küçük harflerle, non-terminal semboller NON_TERMINAL şeklinde temsil edilmeliydi.

Bu dilde yazılan biraz karmaşık bir expression için parse tree çizmemizi istiyordu.

2. Sözel sorulardı.

- **type inference ve type checking in farkı nedir?**

Type Checking looks at the body of each function and use declared types of identifiers to check agreement. Type Inference looks at the code without type information and figure out what types could have been declared

- **C++ adt yi nasıl sağlar ve bunun module'den farkı nedir?**

C++ handles it with namespaces. A module is a way which is used to organize the code in separate files and can execute in their local scope, not in the global scope. A namespace is a way which is used for logical grouping of functionalities with local scoping.

- **C++ ve Java arasındaki farklar nelerdir? (Derste bahsettiği view varmış user view falan galiba o şekilde değerlendirilmesini istiyormuş)**

3. Prolog sorusu. Even_between(1,6,x) sorgusu, (2,4,6) döndürmeliydi. Even_number sorgusu yazmamızı istiyordu, integer(x) ve less than sorgusunun zaten yazılmış olduğunu varsayarak yapıcaktık.

Bu kod için unification ve resolution'ı adım adım göstermemizi istemişti.

```
even_between(A, B, X) :-  
    A <= B,  
    integer(A), integer(B),  
    (    A mod 2 == 0, X = A  
    ;    A1 is A + 1, even_between(A1, B, X)  
    ),  
    less_than(X, B).
```

4. Clojure sorusu. Sonsuz basamaklı olacak, string şeklinde verilecek sayılarımız vardı. Bunlar için bir kütüphane yazmamızı istemişti.

Add_int, add_float, sub_int, sub_float, mult_int, mult_float, ... , sort, less than şeklinde fonksiyonları verip neler döndüreceğinden bahsetmişti. Bazı fonksiyonları bazılarını kullanmadan yazmamızı istemişti... Örneğin mult_int implement ederken add_int kullanmayın şeklinde...

2017 Vize

1. Left regular gramerlerin hangi formda olduğunu anlatıp örnekleme (3pts)

$S \rightarrow B00 \mid S11$
 $B \rightarrow B0 \mid B1 \mid 0 \mid 1$

S,B are non terminals. 0, 1 are terminals.

- All production rules have the form $A \rightarrow w$ or $A \rightarrow Bw$ where A, B are non-terminal symbols, w is a terminal symbol

2. Statement ve expression arasındaki fark nedir (3pts)

Assignments are not expected to return a value. They have side effects. Expression is reverse of it. Without expressions, we could not perform any calculations, without assignments we could not store values, and without variables, we could not keep track of the data of our program.

3. Programlar neden parçalı fonksiyon tanımlar? (3pts)

Partial function $f: A \rightarrow B$ is a subset $f \subseteq A \times B$ with – If $x, y \in f$ and $x, z \in f$ then $y=z$ (single-valued). Computability.

4. Lisp gramer için BNF:

Operators : *, +, -, /, ^

+ 2 ya da daha fazla parametre alır

- 1 ya da 2 parametre alır

*** 2 ya da daha fazla parametre alır**

/, ^ yalnızca 2 parametre alır

(15 pts)

5. Binding nedir ? Static vs dynamic binding with respect of reliability and writability

A variable binding is the association of a variable name with a value or an object in a program. It is the process of connecting a variable to a specific value or memory location, so that the variable can be used to refer to the value or object.

Static binding, also known as compile-time binding, is the process of binding a variable to a value or an object during the compilation of a program. In static binding, the variable is bound to its value or object before the program is executed, and the binding cannot be changed during the execution of the program.

Dynamic binding, also known as runtime binding, is the process of binding a variable to a value or an object during the execution of a program. In dynamic binding, the variable is bound to its value or object at runtime, and the binding can be changed during the execution of the program.

When it comes to reliability, static binding is generally considered more reliable than dynamic binding. Because the bindings are determined at compile-time, the compiler can catch type errors and other issues during the compilation process, making it more difficult for bugs to slip through into the final program.

Regarding writability, dynamic binding is generally considered more writable than static binding. Because the bindings are determined at runtime, it allows to change the behavior of a program at runtime, and to use the same code with different types of objects, which is useful for example in polymorphism.

6. C ve Java'nın tip sistemlerini karşılaştırın. (10 pts) (strongly or weakly typing olarak soruyordu)

Strongly typed languages are those in which variables, functions and expressions are strongly associated with a specific data type, and the type of a variable must be explicitly declared before it can be used. The type of a variable cannot change during the execution of the program. Strongly typed languages are considered more robust and less error-prone, as they can catch type errors at compile-time and prevent them from causing issues at runtime.

Weakly typed languages, on the other hand, are those in which variables and expressions are not strongly associated with a specific data type. The type of a variable can change during the execution of the program. In these languages, variables may be implicitly converted from one data type to another, and type checking is often done at runtime.

7. Hexadecimal sayıları gösterebilmek için bir DFA tanımlamamızı istiyordu. (7 pts)

8. Hexadecimal çift basamaklı sayıları gösterebilmek için de bir DFA istiyordu. (8 pts)

CLOJURE SORULARI

- 1. Parametre olarak liste alan bir fonksiyon yazın. İç içe listeler olabilir. Bu listeyi ters çevirip(reverse) return etmemizi istiyordu. Ex: (1, (2, 3)) -> ((3, 2), 1) (10pts)**

```
(defun reverse-list (lst)
  (if (null lst)
      nil
      (append (reverse-list (reverse (cdr lst)))
                (list (if (listp (car lst))
                          (reverse-list (car lst))
                          (car lst)))))))
```

2. Listedeki bir elemanın kaç kere tekrar ettiğini bulan bir program yazın. İç içe listeler olabilir. Ex: (1, (3, 5), (3, 7, 8), 2) -> 3 occurs 2 times (10pts)

```
(defun count-element (element lst)
  (if (null lst)
      0
      (+ (if (eql element (car lst)) 1 0)
         (if (listp (car lst))
             (count-element element (car lst))
             0)
         (count-element element (cdr lst)))))
```

3. Verilen string palindrome mudur bulan bir kod. Ex: (12321) (15pts)

```
(defun is-palindrome (string)
  (if (<= (length string) 1)
      t
      (and (char= (first string) (last string))
            (is-palindrome (subseq string 1 (- (length string) 1))))))
```

21-22 Vize-mazeretFinal Bohça

1. Explain why applicative order evaluation is not used when side effects are allowed in an expression or a function.

Applicative order evaluation is a strategy for evaluating the expressions in a program, where the arguments to a function are fully evaluated before the function is applied to them. This means that all the side effects (if any) of the argument expressions are performed before the function is applied.

When side effects are allowed in an expression or a function, the order in which the expressions are evaluated can affect the outcome of the program. For example, if a function has a side effect of modifying a global variable, and that function is called with two arguments, the order in which the arguments are evaluated would affect the final value of the global variable.

When side effects are allowed in a program, the order of evaluation can affect the outcome of the program, and there is no way to determine which order the expressions should be evaluated in, to get the correct result. Therefore, applicative order evaluation is not used when side effects are allowed in an expression or a function.

2. What type of errors do occur in shift reduce parser. How do we resolve them?

If there is a choice with 2 options, error occurs in SRP.

If it is legal to shift or reduce, shift-reduce conflict.

If it is legal to reduce by two different productions, reduce-reduce conflict.

To resolve shift-reduce conflicts, we use precedence and associativity information. To resolve reduce-reduce conflicts, we modify the grammar to remove the ambiguity

3. Verilen production rule için Shift-Reduce Parser yapın. 8p

4. Smalltalk ve javascript arasındaki temel farklar nelerdir? Hangi durumda hangi dili kullanırdınız? 3p

5. Prolog ile sudoku çözen algoritma yazın. 12p

6. Type Checking ve Type inference arasındaki farklar nelerdir? 3p

7. Kuralları verilen dil neden fonksiyonel bir dildir? Dilin ADT'yi sağlaması için neler eklenebilir?

8. Lisp, liste ile graph yapısı verilmiş.

-A şıkkı: Graphta, verilen iki node arası path varsa true yoksa false döndürün. 15p

-B şıkkı: verilen iki graph eşitse true değilse false döndürün. 20p

2015 Final

1. Module ile namespace arasındaki fark nedir?

A module is a way which is used to organize the code in separate files and can execute in their local scope, not in the global scope. A namespace is a way which is used for logical grouping of functionalities with local scoping.

2. Overloading ve polymorphism nedir farkları nelerdir?

Overloading refers to the ability of a function or operator to have multiple implementations, each with a different set of parameters. This allows the same function or operator to be used with different types of input or with different numbers of input arguments. For example, in C++, an operator like "+" can be overloaded to work with different types like integers, floating point, and user-defined classes.

Polymorphism refers to the ability of a function or operator to have a single implementation, but to be able to work with different types of input. This is achieved through the use of virtual functions, interfaces, or base classes. For example, in C++, a virtual function like "draw()" can be overridden in different derived classes, like "Circle" and "Rectangle", to provide different implementations depending on the type of the object. In summary, overloading allows a single function or operator to have multiple implementations, each with a different set of parameters, and polymorphism allows a single function or operator to have a single implementation but to be able to work with different types of input. While overloading is related to the number of arguments and types of arguments, polymorphism is related to the type of the object, so that the same function/method can be applied to different types of objects.

3. Axiom, horn clause, query nedir. Örnek ver.

- Axiom: An axiom is a statement that is assumed to be true, and serves as the foundation for building logical proofs. In logic, an axiom is a statement that is taken to be true without any need for proof. For example, in first order logic, Peano's axioms are a set of five axioms that are used to define the natural numbers, which are assumed to be true without any need for proof.
- Horn Clause: A Horn clause is a logical statement in the form of "if A then B" where A is a conjunction of literals and B is a single literal. Horn clauses are a subset of first-order logic clauses and are important in the field of automated reasoning and knowledge representation. An example of a Horn clause is "if it's raining and I have an umbrella then I will take my umbrella."
- Query: A query in logic programming is a statement used to ask the system for a specific piece of information. Queries are used to extract information from the knowledge base. A query is a logical statement that is used to extract information from a knowledge base, usually written in a declarative way. For example, in Prolog, a query can be a question like "Is X a parent of Y?" or "What is the capital of country Z?".

In summary, An axiom is a statement that is assumed to be true, Horn clause is a logical statement in the form of "if A then B" and query is a statement used to ask the system for a specific piece of information.

4. **C++daki namespace kavramı..**
5. **6 dilin language typeini yaz. Haskell, Java, Prolog, C, C++, Smalltalk.**

- Haskell: Haskell is a functional programming language.
- Java: Java is an object-oriented programming language.
- Prolog: Prolog is a logic programming language.
- C: C is a procedural programming language.
- C++: C++ is a multi-paradigm programming language, it has both procedural and object-oriented features.
- Smalltalk: Smalltalk is an object-oriented programming language.

Lisp SORULARI

6. **Sıralı olan bir listeye bir eleman ekle. Liste sıralı olmaya devam etsin.**

```
(defun insert-sorted (element lst)
  (if (null lst)
      (list element)
      (if (> element (car lst))
          (cons (car lst) (insert-sorted element (cdr lst)))
          (cons element lst)))))
```

7. **Bir listeyi sırala**

8. **Graph DFS**

2016 Final

- Lisp: Fraction sayılar için +,-,*,/ fonksiyonlarının yazılması
- Lisp: 2 tane listenin ortak elemanlarını döndüren fonksiyon
- axiomatic spesification sorusu vardı. Ders slaytlarında örneği var. Bir yapı için syntactic, semantic spec. yazılması istenmişti.
- dynamic lookup neden daha iyidir?

different code for different objects

- OOP differences between prototype based and class-based

in prototype based feature of inheritance implemented via cloning. it's maybe a quicker way to turn around.

1. How can you show if a given CFG is ambiguous?

A context-free grammar (CFG) is considered to be ambiguous if it generates more than one parse tree for the same string. There are a few ways to show that a given CFG is ambiguous:

2. Explain explicit heap-dynamic variables.

Allocated and de-allocated by explicit directives. Specified by programmer. Takes effect during execution.

Advantages: provides for dynamic storage management (flexibility)

Disadvantages: inefficient by comparison (cost of reference), reliability.

3. How is cartesian products relevant in types?

In type theory, the Cartesian product is a fundamental concept that is used to combine two or more types into a new type. The Cartesian product of two types, A and B, is denoted as $A \times B$, and represents the set of all ordered pairs (a, b) where a is an element of A and b is an element of B.

The Cartesian product is often used to define the product type in programming languages. The product type allows for the representation of complex data structures that consist of multiple values of different types. For example, the Cartesian product of two types A and B can be used to represent a point in 2D space, where A represents the x-coordinate and B represents the y-coordinate.

4. What are discriminated unions?

A discriminated union is a union data structure that holds various objects, with one of the objects identified directly by a discriminant. The discriminant is the first item to be serialized or deserialized. A discriminated union includes both a discriminant and a component.

5. Main differences between expression and statements.

Statements are not expected to return a value. They have side effects. Expression is reverse of it.

6. What is exception binding?

Binding an exception occurrence to an exception handler. At a unit level or at a higher level.

7. What is type checking? What are two type checking mechanism do you know?

The translator checks if data types are used correctly.

Type Inference: Infer the type of an expression, whose data type is not given explicitly.

Type Equivalence :Compare two types, decide if they are the same.

8. What is type inference?

Type Inference: Infer the type of an expression, whose data type is not given explicitly.

9. Why does C have separate compilation?

Because modules can be imitated using separate compilation

10. What is difference between macro and call by reference?

Similar to call-by-name, but uses textual substitution rather than capture-avoiding substitution.

11. What are name spaces?

a namespace is a set of signs (names) that are used to identify and refer to objects of various kinds. A namespace ensures that all of a given set of objects have unique names so that they can be easily identified.

12. What are class variables and methods?

In object-oriented programming, a class is a blueprint for creating objects (also known as instances) that have certain properties (also known as attributes) and can perform certain actions (also known as methods).

13. What are object instance methods and variables? Provide examples.

An instance variable is a variable which is declared in a class but outside of constructors, methods, or blocks. Instance variables are created when an object is instantiated, and are accessible to all the constructors, methods, or blocks in the class.

Instance method are methods which require an object of its class to be created before it can be called. To invoke a instance method, we have to create an Object of the class in which the method is defined

14. Diff between class and object in smalltalk.

In Smalltalk, a class is a blueprint for creating objects, it defines the properties and methods that the objects created from that class will have. An object is an instance of a class, it has its own unique state, and it can respond to messages defined in its class. Classes in Smalltalk are also objects and are instances of the class "Metaclass", which defines how classes should behave.

15. How are interfaces realized in c++.

In C++, interfaces are typically realized through abstract classes. An abstract class is a class that contains one or more pure virtual functions (also known as abstract methods). A pure virtual function is a virtual function that has no implementation in the base class and must be implemented by any derived classes.

Look 2016 Final. BNF question.

Lists in lisp can be considered an ADT. Lisp has cons car cdr nil in lists. A list can be of any basic type.

16. How can we specify ADT?(consider two main components for the specification)

Syntactic specification (signature, interface):
the name of the type, the prototype of the operations

Semantic specification (axioms, implementation): guide for required properties in implementation mathematical properties of the operations

17. Write the first part of the specification for the data type described above.

18. Same question with 17, second part.

19. How would you implement such an ADT in C++. Describe the data representation as well as other implementation details.

20. Assume that you have a pl that does type inference. How would type inference work for the ADT above.

- When the cons operation is called with a specific element x and a list L as arguments, the type of the element x would be inferred and the type of the resulting list would be a list of that type.
- When the car operation is called with a specific list L as an argument, the type of the first element of the list L would be inferred.
- When the cdr operation is called with a specific list L as an argument, the type of the resulting list would be the same as the type of the input list L, but without the first element.
- When the nil is used, it would be inferred as the end of the list.

21. Dynamic lookup is important concept in OOP languages. Compare c++ and java in terms of lookup strategies. Your evaluation should consider classes, objects and class and object methods and variables.

In java, java uses dynamic linking and interfaces. Due to that, Java's dynamic lookup is slower than C++. Java uses static typing, it is very efficient than c++.

both C++ and Java support dynamic lookup, but the way in which they handle it can differ. C++ uses virtual function tables (vtable) and virtual function pointers (vptr) to achieve dynamic lookup, while Java uses method overriding and dynamic method dispatch. C++ also allows both static and non-static members for class methods and variables, while Java uses the concept of static methods and variables.

Answer next questions in the context of functional programming.

What is an expression

Programs are executed by evaluating expressions. Thus semantics are defined by **evaluation rules** of expressions.

What are eager and lazy functions

Eager functions, also known as strict or strict functions, are functions that evaluate their arguments and perform the computation immediately, before returning a result. Once an eager function is called, it will always return the same result for the same input. Eager functions are said to be strict in the sense that they always evaluate their arguments, regardless of whether their results are used.

On the other hand, lazy functions, also known as non-strict or non-strict functions, are functions that delay the evaluation of their arguments and computation until the result is needed. Lazy functions only evaluate their arguments when their results are used, and they can return different results for the same input depending on when they are called.

When would lazy functions be useful.

lazy functions are useful in many situations where the evaluation of the function's arguments and computation can be delayed until the result is needed, such as infinite data structures, large data sets, conditional evaluation, debugging and testing, and pure functional programming.

What is a lambda expression when it is used.

In programming languages, a lambda expression (also known as anonymous function or closure) is a small, inline, and anonymous function that can be passed as an argument to other functions or assigned to a variable. Lambda expressions are typically used to define small, one-time-use functions that are not intended to be reused or named.

To define small, one-time-use functions that are not intended to be reused or named

- To define function arguments for higher-order functions
- To define function arguments for functional data structures, such as lists and maps

Compare java and lisp for their expression evaluation strategies

Java-eager Lisp-Lazy

Look 2016 final from final1 file in drive.

Lisp

Pairup function

```
(defun pairup (x y)
  (if (equal nil x)
      (progn
        (if (equal nil y)
            '()
            (cons (cons -1 (car y)) (pairup x (cdr y))))
        )
      (progn
        (if (equal nil y)
            (cons (cons (car x) -1) (pairup (cdr x) y))
            (cons (cons (car x) (car y)) (pairup (cdr x) (cdr y))))
        )
      )
  )
)
```

Swap2 function

```
(defun swap-consecutive-elements (lst)
  (if (or (null lst) (null (cdr lst)))
      lst
      (cons (cadr lst) (cons (car lst) (swap-consecutive-elements (cddr
lst))))))
```